EEL 5764: Computer Architecture

Lecture 9: RISC-V Instruction Set Architecture (ISA)

Sandip Ray

Warren B. Nelms Endowed Professor

Department of Electrical & Computer Engineering, University of Florida | Office: MALA 4131

Required Reading: H&P Appendix A, P&H Chapter 2

ISA as a System Contract: The Hardware-Software Interface

THE SOFTWARE CONTRACT

- The Programmer's visible state: Defines the available processor state, including register layouts, program counter, and memory addressing spaces.
- Orthogonal instruction classes: Defines instructions, operands configuration, arithmetic logic operations, data transfer, and control flow semantics.
- Decoupling software from hardware: A compiler targets the ISA, and any compliant processor executes it safely. Software compiled for the ISA runs on any microarchitecture executing that contract.

WHY RISC-V FOR GRADUATE RESEARCH?

- Free, open, and extensible: Offers an open-standard architecture. Research groups can design and plug in custom domain-specific acceleration extensions without licensing penalties.
- Clean modular design: Consists of a mandatory, highly streamlined base integer instruction set (RV32I/RV64I) paired with modular standard extensions (M, A, F, D, C, V).
- No legacy baggage: Eliminates the complex instruction decoding hardware overhead of x86-64, allowing architects to spend their silicon budget on execution and energy optimization.

Design Philosophy: RISC vs. CISC Trade-offs

CISC (COMPLEX INSTRUCTION SETS)

- Variable instruction length: Instructions span from 1 to 15 bytes (e.g., in x86), requiring highly complex physical decoding logic.
- Complex memory-to-memory operations: Allows instructions to directly perform arithmetic on memory locations (e.g., add value to address in-place).
- Minimize compiler footprint: Designed in an era when memory was extremely expensive, aiming to cram maximum semantic meaning into each instruction.
- Physical cost: Leads to massive hardware decoder area overhead, high power leakage, and slower operating frequency limits.

RISC (REDUCED INSTRUCTION SETS)

- Fixed instruction length: Every instruction is strictly 32-bit (in standard base RV32/RV64), enabling high-speed, parallel instruction alignment checking.
- Load-Store Architecture: Hardware logic restricts memory access to explicit load and store instructions. All arithmetic is register-to-register.
- Simplify decoding: Keeps hardware decoders simple and fast, enabling deeper pipeline execution, higher clock frequencies, and lower dynamic power.
- Outcome: Shifts complexity from physical hardware circuits into compiling toolchains, which is an extremely profitable trade-off today.

The RISC-V Programmer-Visible State

THE REGISTER FILE SPACE

- General-purpose registers: Contains exactly 32 general-purpose registers, designated from x0 through x31.
- Register width: Configured as 32 bits wide in RV32, and 64 bits wide in RV64 (often referred to as 'XLEN' bits).
- The hardwired constant zero: Register x0 (zero) is physically grounded to 0. Reading x0 always returns 0, and any write to x0 is discarded.
- Register utility: Fast, low-latency temporary storage located directly inside the processor datapath, executing in less than one clock cycle.

MEMORY & PROGRAM CONTROL

- Byte-addressable memory: Memory address indices target individual 8-bit bytes. The address space spans 2^32 bytes (RV32) or 2^64 bytes (RV64).
- The Program Counter (PC): An independent register that holds the absolute memory address of the instruction currently being executed.
- Word alignment: Instructions must be aligned to 4-byte boundaries (addresses are multiples of 4). If the compressed 'C' extension is supported, alignment is relaxed to 2-byte boundaries.
- Hardware-enforced state: PC is updated automatically after each fetch, or modified by control-flow branches and jumps.

RISC-V Register Names and ABI Conventions

TEMP & SAVED REGISTERS

- ra (x1) - Return Address: Holds the return address for function subroutines (updated by jal/jalr jump-and-link instructions).
- sp (x2) - Stack Pointer: Points to the current top of the active execution stack frame.
- t0 - t2 (x5-x7), t3 - t6 (x28-x31): Caller-saved temporary registers. Can be overwritten freely by a subroutine without saving.
- s0 / fp (x8), s1 (x9), s2 - s11 (x18-x27): Callee-saved registers. Subroutines must save these on the stack before using, and restore them before returning.

FUNCTION ARGUMENTS & CONSTANTS

- a0 - a1 (x10-x11): Used to pass the first two function call arguments, and to return values back to the caller.
- a2 - a7 (x12-x17): Used to pass up to six additional function call arguments. Excess arguments must be passed via the stack.
- gp (x3) & tp (x4): Global and Thread pointers, used to coordinate global variables and multi-threaded memory spacing.
- zero (x0): The ABI constant zero register, which simplifies register-register operations in compiler layouts.

Orthogonal Instruction Formats: Fast Hardware Decodes

REGISTER FIELD ALIGNMENT

- Structural orthogonality: Every instruction format keeps register specifiers in the exact same physical bit positions across all configurations.
- Parallel Register Fetch: The microarchitecture can fetch rs1, rs2, and rd in parallel with analyzing the opcode. No sequential decoding is required.
- Fixed Bit Slices:
- • rs1 (Source register 1) is always bits [19:15]
- • rs2 (Source register 2) is always bits [24:20]
- • rd (Destination register) is always bits [11:7]

THE 6 BASE FORMAT TYPES

- R-Type (Register-Register): Used for computational arithmetic, shifting, and logical operations with three register operands.
- I-Type (Immediate/Load): Used for register-immediate calculations, memory load instructions, and indirect jumps.
- S-Type (Store): Used to write register values to memory offsets.
- B-Type (Branch): Used for conditional jumps (features scrambled immediate bits to keep register positions locked).
- U-Type (Upper Immediate): Loads 20-bit upper immediates.
- J-Type (Jump): Unconditional jumps with wide PC-relative targets.

R-Type Format: Register-Register Operations

R-TYPE BIT LAYOUT

- funct7 [31:25] (7 bits): Secondary instruction detail field, used to distinguish sub-operations (e.g., ADD vs. SUB).
- rs2 [24:20] (5 bits): Second source register input.
- rs1 [19:15] (5 bits): First source register input.
- funct3 [14:12] (3 bits): Primary operation identifier (e.g., ADD/SUB, SLL, SLT, XOR, SRL, OR, AND).
- rd [11:7] (5 bits): Target destination register.
- opcode [6:0] (7 bits): Standard operation class identifier (strictly coded as 0110011 for base R-type).

COMPUTATIONAL OPERATIONS

- Addition: add x5, x6, x7 -> x5 = x6 + x7 (funct7 = 0000000, funct3 = 000).
- Subtraction: sub x5, x6, x7 -> x5 = x6 - x7 (funct7 = 0100000, funct3 = 000).
- Bitwise XOR: xor x12, x13, x14 -> x12 = x13 ^ x14 (funct7 = 0000000, funct3 = 100).
- Shift Left Logical: sll x2, x3, x4 -> Shifts x3 left by amount in x4, storing result in x2 (funct7 = 0000000, funct3 = 001).

I-Type Format: Immediate and Load Operations

I-TYPE BIT LAYOUT & THE 12-BIT LIMIT

- imm[11:0] [31:20] (12 bits): Signed immediate constant value.
- rs1 [19:15] (5 bits): Source register input.
- funct3 [14:12] (3 bits): Specifies arithmetic or logical type.
- rd [11:7] (5 bits): Target destination register.
- opcode [6:0] (7 bits): Coded as 0010011 for arithmetic, or 0000011 for loads.
- The 12-bit signed range: Immediates are sign-extended to 32 bits, limiting constant values from -2048 to +2047.

ARITHMETIC IMMEDIATE OPERATIONS

- Add Immediate: addi x10, x11, 45 -> x10 = x11 + 45 (funct3 = 000, opcode = 0010011).
- And Immediate: andi x12, x13, 0xF -> x12 = x13 & 15 (funct3 = 111).
- Set Less Than Immediate: slti x5, x6, 100 -> x5 = 1 if x6 < 100 else 0 (funct3 = 010).
- Synthesizing copies: addi x5, x6, 0 -> Dispatches register copy x5 = x6. This is the official compiler representation of MV.

S-Type Format: Memory Store Operations

THE SPLIT-IMMEDIATE DESIGN

- The Split Offset: The 12-bit immediate offset is split across two fields: imm[11:5] (bits [31:25]) and imm[4:0] (bits [11:7]).
- Why split the immediate? To preserve the exact physical positions of the source registers: rs1 is kept at [19:15] and rs2 is kept at [24:20].
- Hardware simplification: Eliminates the need for a multiplexer on the register output lines, saving physical silicon routing and decode latency.
- Opcode identifier: Strictly coded as 0100011 for store operations.

STORE INSTRUCTION EXAMPLES

- Store Word: sw rs2, offset(rs1) -> Write 32 bits of rs2 into memory address rs1 + sign-extend(offset).
- Store Byte: sb rs2, offset(rs1) -> Write lower 8 bits of rs2 into memory address.
- Store Halfword: sh rs2, offset(rs1) -> Write lower 16 bits of rs2 into memory address.
- Memory Addressing Mode: Displacement addressing (base register + constant offset) is the only memory mode supported by standard RISC-V.

Data Transfer Mechanics: Loads and Stores

MEMORY ADDRESSING & ALIGNMENT

- Load-Store Model: The CPU cannot perform arithmetic operations directly on main memory values. Memory variables must be explicitly loaded into registers first.
- Memory Address Calculation: Effective Address = Base Register (rs1) + Sign-Extended 12-bit Offset.
- Aligned accesses: Word-addressable transactions should target 4-byte boundaries, while halfwords should target 2-byte boundaries. Unaligned accesses are supported in some designs but incur penalty cycles.

SIGN EXTENSION ON LOADS

- Load Byte (lb) - I-Type: Loads 8 bits and sign-extends the highest bit across the remaining upper 24 bits of rd.
- Load Byte Unsigned (lbu) - I-Type: Loads 8 bits and zero-extends the remaining upper 24 bits of rd. Very critical for processing character strings or unsigned arrays.
- Load Halfword (lh / lhu) - I-Type: Loads 16 bits with sign-extension (lh) or zero-extension (lhu) to fill the 32-bit destination register rd.

B-Type Format: Conditional Branching

CONDITIONAL CONTROL FLOW

- B-Type Layout: Conditional branch instructions compare rs1 and rs2, and perform a PC-relative jump if the condition is met.
- Scrambled Immediate: Similar to S-type, the immediate is scrambled (imm[12], imm[10:5], imm[4:1], imm[11]) to keep register fields aligned.
- Halfword Alignment: Because instructions are always 2 or 4-byte aligned, the lowest bit (imm[0]) is always 0. RISC-V does not store this 0 bit, effectively doubling the branch range.
- Branch Range: PC-relative branch targets span +/- 4 Kilobytes of memory.

BRANCH INSTRUCTION SET

- Branch Equal: beq rs1, rs2, Target -> Branch if rs1 == rs2 (funct3 = 000).
- Branch Not Equal: bne rs1, rs2, Target -> Branch if rs1 != rs2 (funct3 = 001).
- Signed Comparisons:
- • blt (Branch Less Than) and bge (Branch Greater Than or Equal)
- Unsigned Comparisons:
- • bltu and bgeu (interprets registers as raw unsigned integers).

U-Type and J-Type Formats: Large Immediates and Jumps

U-TYPE (UPPER IMMEDIATES)

- Target Utility: Used to load large constant values and build absolute 32-bit addresses in memory.
- U-Type Layout: Contains a large 20-bit immediate field (imm[31:12]) and a 5-bit destination register rd.
- Load Upper Immediate: lui rd, immediate -> Places the 20-bit immediate in the upper 20 bits of rd, and zero-fills the lower 12 bits.
- Address Construction: Pair lui with a subsequent addi instruction to construct any full 32-bit constant or pointer in a register.

J-TYPE (UNCONDITIONAL JUMPS)

- Jump and Link: jal rd, offset -> Performs an unconditional jump to PC + sign-extend(offset) and stores return address PC + 4 in rd (typically ra).
- Unconditional Jump Range: PC-relative jumps span +/- 1 Megabyte of memory.
- Indirect Jumps - jalr (I-Type): jalr rd, offset(rs1) -> Jump to absolute address rs1 + offset, storing PC + 4 in rd. Enables dynamic function pointers and subroutine returns (jalr x0, 0(ra)).

Challenge: Reconstructing Assembly to Binary

THE ASSEMBLY CHALLENGE

- Assembly Target Instruction: addi x5, x6, -10
- Objective: Deconstruct this assembly instruction and translate it into its exact 32-bit binary representation and hexadecimal code.
- Instruction Classification: Identify the layout as I-type.
- Bit Fields to Solve:
- • imm[11:0] (12 bits)
- • rs1 (5 bits)
- • funct3 (3 bits)
- • rd (5 bits)
- • opcode (7 bits)

STEP-BY-STEP BINARY ASSEMBLY

- Destination Register rd (x5): binary 00101 (decimal 5).
- First Source Register rs1 (x6): binary 00110 (decimal 6).
- funct3 for addi: binary 000.
- Opcode for OP-IMM arithmetic: binary 0010011 (decimal 19).
- Immediate imm (-10): 12-bit two's complement value.
- +10 = 0000 0000 1010
- Invert & Add 1 = 1111 1111 0110
- Pack fields: 111111110110 | 00110 | 000 | 00101 | 0010011
- Convert to Hexadecimal: 0xFF630293.

Challenge 2: Deconstructing a Compiler Pointer Loop

THE HIGH-LEVEL POINTER LOOP

```
// High-Level C Code
while (save[i] == k) {
    i += 1;
}

// Baseline Variables Mapping:
//   - Loop index 'i' in register x22
//   - Constant search 'k' in register x24
//   - Array base address 'save' in register x25

// Design Constraints:
//   - save is a 32-bit integer array
//   - Each element consumes 4 bytes of memory
//   - Addressing requires scaling offset by 4
```

COMPILED RISC-V ASSEMBLY

```
Loop:
  slli x10, x22, 2     # x10 = i * 4 (offset)
  add  x10, x10, x25   # x10 = addr of save[i]
  lw   x9, 0(x10)      # Temp register x9 = save[i]
  bne  x9, x24, Exit   # if save[i] != k, exit
  addi x22, x22, 1     # i = i + 1
  jal  x0, Loop        # Jump to Loop
Exit:

// Key Takeaways:
//   1. slli (shift left logical immediate) scales
//      index to byte address boundary (i * 4).
//   2. jal x0 targets x0 to discard return point,
//      synthesizing an unconditional jump.
```

Hands-On Compilation: GCC Assembly & Disassembly

GENERATING RISC-V ASSEMBLY FROM C

```
// High-Level C Code (test.c)
int add_val(int a) {
    return a + 5;
}

// Compile to RISC-V Assembly (.s)
riscv64-linux-gnu-gcc -S -O2 test.c -o test.s
```

Key compiler flags:

- -S : Directs compiler to output assembly code.
- -O2: Enables optimization to compact instructions.

ASSEMBLING & DISASSEMBLING CODE

```
// Assemble into Object Binary (.o)
riscv64-linux-gnu-gcc -c test.s -o test.o

// Disassemble back to Assembly with Hex
riscv64-linux-gnu-objdump -d test.o
```

Key assembly & tool metrics:

- -c : Assembles assembly into compiled object binary.
- -d : Disassembles the machine code bytes back.
- objdump: Traces exact 32-bit machine hex byte codes alongside instruction assembly mnemonics.